

A Deep Learning-Based In Situ Analysis Framework for Tropical Cyclogenesis Prediction

Abir Mukherjee[§]

Citrix R&D India Pvt. Ltd.
Bangalore, India

Preeti Malakar

Department of Computer Science and Engineering
Indian Institute of Technology Kanpur, India

Abstract—Tropical cyclone is one of the most violent natural disasters causing massive devastation. Accurate forecasting of cyclones with high lead times is an important problem. We propose a framework to predict tropical cyclogenesis (i.e. cyclone formation). This framework executes along with a parallel weather simulation model (WRF) and analyzes the simulation output as soon as they are generated. Our framework has two major components – a trigger function and a deep predictive model. The trigger function acts as a basic filter to identify cyclones from non-cyclones. The proposed deep learning model is based on convolutional neural networks (CNNs). The best track data from Indian Meteorological Department (IMD) is used as a reference for labeling data points into disturbances and tropical cyclones. The framework achieves a probability of detection (POD) value of approximately 95% with a false alarm ratio (FAR) of 21.69% overall. The predictions made by the framework have a lead time of up to 150 hours from the time that a disturbance transforms into a tropical cyclone.

Index Terms—tropical cyclones, cyclogenesis, deep learning, CNN, ConvLSTM, trigger, WRF

I. INTRODUCTION

Natural calamities adversely affect the human population causing loss of lives and economic losses [1], [2]. Tropical cyclone is one of the most devastating natural events. Tropical cyclones (TCs) are storms formed in the tropical region, having a warm low-pressure center and strong circular winds. Accurate predictions of TCs with good lead times (i.e. early predictions) can help in timely evacuations. This helps in preventing loss of lives and in reducing economic damages [3] caused due to the natural disasters. A significant progress has been made in the prediction of tropical cyclones. This progress can be attributed to both the development of better theoretical models [4] as well as to the improvement of high performance computing technologies. Modern predictive methods such as machine learning and deep learning [5] are now being integrated into prediction systems.

The two major components of a tropical cyclone warning system are – a cyclogenesis prediction system and a cyclone track prediction system. The cyclogenesis prediction system is responsible for differentiating between developing and non-developing tropical disturbances. Only a developing tropical disturbance will mature into a tropical cyclone [6]. Predicting cyclogenesis with high accuracy and a high lead time is complicated due to the incomplete understanding of the actual

genesis process. In this work, we analyze the simulation output data for cyclogenesis predictions concurrently with the parallel simulation executions using insights about the important meteorological variables from prior works [7], [4]. Concurrent analysis of simulation output enables higher lead times. We propose an integrated framework for predicting tropical cyclogenesis events *on-the-fly* using deep learning techniques. The prediction framework executes along with the simulations and makes new predictions whenever new data is available. Weather simulation domains are typically large; the output generated from these high performance simulations is of the order of gigabytes for finer resolutions. Thus we designed a lightweight trigger mechanism that helps in pruning the data and acts as a filter for the deep learning model.

The three main contributions of this paper are (1) a deep learning model for predicting cyclogenesis events, (2) a novel trigger function for identifying regions of interest over the input domain that are fed to the deep learning model and (3) an end-to-end framework that incorporates the above two to make *on-the-fly* cyclogenesis predictions. The framework interfaces with an ongoing Weather Research and Forecasting (WRF) [8] simulation and analyzes the output as the simulation progresses. We used 80:20 split of 60,000 datapoints (contain cyclonic and non-cyclonic data) for the DL model. The framework achieves a probability of detection (POD) value of about 95% with a false alarm ratio (FAR) of 21.69%. We achieved lead times (i.e. advanced forecast) of up to 150 hours for the predictions.

II. BACKGROUND

A. Tropical Cyclones

Cyclones are strong circular winds formed around low pressure areas. Cyclones that occur in the region between the Tropic of Cancer and the Tropic of Capricorn, generally formed in the warm tropical oceans, are termed as tropical cyclones. A tropical cyclone has a warm low-pressure core, called the eye, and a high-pressure region, of intense circular winds, surrounding the eye. Tropical cyclones broadly have three stages – the early stage or genesis stage, the mature stage and the dissipation stage. Initially, they start out as weak tropical disturbances in the genesis stage. These may intensify into tropical cyclones, resulting in a successful genesis. The genesis of TCs is affected by both internal and external factors

[§]Work done when the author was affiliated with IIT Kanpur

and is partially understood. Gray et al. [9] describe six necessary, yet insufficient, parameters that affect the cyclogenesis. Other environmental factors such as interseasonal oscillations, monsoon troughs, intertropical convergence zone and internal factors such as convective evolution and thermodynamic evolution also affect the cyclogenesis process [4]. The parameters used in our deep learning model are based on these prior works as mentioned in Section III. Indian Meteorological Department (IMD) [10] classifies tropical disturbances into eight categories based on their wind speeds, ranging from 31 km/h for depressions to 221 km/h for super cyclonic storms.

B. Weather Research and Forecasting Model

Numerical weather prediction (NWP) systems are used for simulating weather phenomena. These systems simulate equations describing the behaviour of the atmosphere. Weather Research and Forecasting (WRF) model [8] is a mesoscale numerical weather prediction system, used for both atmospheric research and operational forecasting. We used WRF to simulate real data obtained from meteorological observations [11]. WRF simulates and outputs various meteorological variables over the entire period of simulation (which may be thousands of time steps). The output is stored in history output files as per the output frequency specified by the user. Table I shows some of the relevant WRF output variables that have been used in our work for trigger function and the deep learning model. This selection was made based on prior works [12], [9].

WRF Output Variable	Description	Units
SLP	Sea-level pressure	Pascal (Pa)
SST	Sea surface temperature	Kelvin (K)
U	x-wind component	ms^{-1}
V	y-wind component	ms^{-1}
W	z-wind component	ms^{-1}
QVAPOR	Water vapor mixing ratio	—
P	Perturbation pressure	Pascal (Pa)
PB	Base state pressure	Pascal (Pa)
U10	U at 10-meter	ms^{-1}
V10	V at 10-meter	ms^{-1}
AVO	Low-level absolute vorticity	s^{-1}

TABLE I: WRF output variables used in our framework.

Figure 1 shows the visualization (using vector glyphs) of wind vectors of cyclone Hudhud. This visualization gives us insight into the state of an atmospheric variable (wind velocity) during a tropical cyclone. The history output file is generated at intervals defined by the `history_interval` parameter (typically set to 3 hours) in the WRF input configuration file. The simulation output is further used by visualization tools such as VAPOR [13] and VisIt [14]. The output is further analyzed (e.g. using ML) for insight about the simulation. We directly feed this simulation output data into our deep learning model to determine the occurrence of cyclones. This is only done for the output that are filtered by our trigger function (i.e. there may be some possibility of cyclone formation).

C. Convolutional Neural Networks

Convolutional Neural Networks (CNNs) [15] are neural networks that were designed to process data with grid structure.

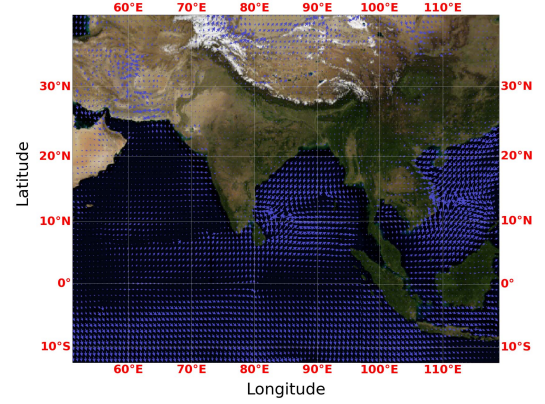


Fig. 1: Visualization of cyclone Hudhud that hit eastern India in October 2014. Wind velocity vectors concentrated around $(10^{\circ}N, 85^{\circ}E)$ depicts the eye of the cyclone.

CNNs have proven successful in capturing spatial relations across grids, such as images, and generalizing over them. Convolution is the primary operation in these neural networks. Figure 2 shows the structure of a CNN for a 2D input data. The convolution operation is performed by sliding a kernel over the input grid. The kernel is a grid of weights, usually much smaller in size than the input grid. The calculated value of convolution may be passed through an activation function. Another type of operation is pooling function which is used to calculate summary statistics. The grids of output values

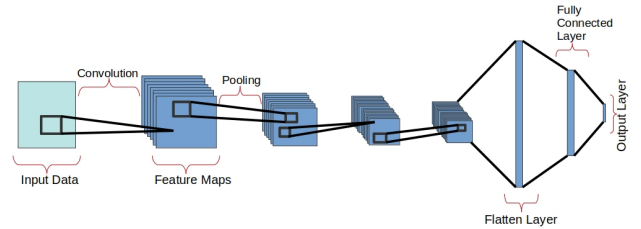


Fig. 2: Example of a Convolutional Neural Network (CNN)

generated by these operations are called feature maps. In a deep convolutional neural network, these feature maps are further fed into different layers of operations. The weights of the kernel are either obtained through training or are predefined using some specific domain knowledge. Computational complexity of learning using CNN is significantly lower than a fully connected network. CNNs are also invariant to small translations, thus improves prediction power of the network.

III. RELATED WORK

One of the first major studies on the process of tropical cyclogenesis described six necessary but insufficient environmental parameters for tropical cyclogenesis [7]. These parameters were used to define a Genesis Potential Index (GPI) ([9], [12]) for predicting seasonal tropical cyclogenesis. Other influencing factors were later discovered, such as the effect of Madden-Julian Oscillation [16], the effect of breakdown of

intertropical convergence zone [17] and convective evolution [18]. A key to building better prediction system lies in the improved understanding of the underlying processes. A lot of progress [4] has been made in the understanding of internal and external processes influencing tropical cyclogenesis. The conditions put forth by Gray et al. [7] formed the basis of our trigger mechanism design. It also influenced our choice of variables for designing the deep learning model.

Halperin et al. [19] provides a comparative analysis of TC genesis prediction performance of various global numerical models. McDonnell et al. [20] describes a Poisson regression model for predicting tropical cyclogenesis. Hennon et al. [21] uses tropical cloud cluster dataset to compute eight large scale predictors. The linear combination of the predictors, that gives the best TC genesis prediction performance, is computed using discriminant analysis. This shows that using multiple variables gives better discriminative power to prediction models. Wind vector data is used in [22] to predict tropical cyclogenesis. This influenced our choice of using wind velocity as a feature in our model. Albert et al. [23] demonstrated the effectiveness of tropical cyclogenesis identification using the Okubo-Weiss parameter [24], [25]. This parameter is derived using relative vorticity indicating that vorticity is a good predictor for cyclones. Thus we included vorticity in our model.

Machine learning (ML) and deep learning (DL) based methods are among the more recent techniques for predicting tropical cyclogenesis. Zhang et al. [26] proposed a decision tree based method for discriminating between developing and non-developing tropical disturbances. A decision tree based ML method is applied over WindSat data by Park et al. [27] for detecting tropical cyclone formation. Zhang et al. [28] uses AdaBoost, a machine learning method, to predict the possibility of a mesoscale convective system developing into a tropical cyclone. Matsuoka et al. [29] uses convolutional neural networks over outgoing longwave radiation data obtained from cloud-resolving global atmospheric simulation to identify tropical cyclones and their precursors. Zhang et al. [30] also uses CNN based deep learning model over gridded satellite and ERA5 reanalysis data to predict tropical cyclogenesis. We proposed a deep learning based predictive model using WRF simulation data in our initial work [31]. The training dataset was built by extracting 15 cyclonic regions and 30 non-cyclonic regions from the simulation output. Using these we trained and tested a ConvLSTM based deep predictive model, and achieved an average accuracy of approximately 77%. In this work, we used a different type of deep learning model, trained on a different and minimal set of variables on a bigger dataset and added a trigger mechanism.

In our work, the prediction model is integrated into an online framework (predictions are made while simulations are running). In contrast to the above models, our prediction model predicts along with the ongoing simulation and analyzes its output as soon as it is available. Our work is based on deep convolutional neural networks, and the model is augmented using a pre-filtering trigger mechanism for analyzing entire simulation data. Triggers [32] have been studied in the recent

past, however, they are application-specific in most cases. We have developed novel triggers, specifically for cyclone detection, to filter the input data that is fed to our deep neural network. The triggers help in reducing the data volume fed to the deep learning model and lowers the false alarm rate.

IV. ONLINE CYCLOGENESIS PREDICTION

In this section, we describe the design of our framework, its components, their interactions and the implementation. We developed an initial model as described in Section IV-A. Next, we refined this to a complete framework to predict cyclogenesis in real-time from simulations. We included triggers to alert; a more robust model is described in Section IV-B.

A. Prologue to the Framework

Our initial model was a deep learning model that was trained over a dataset of WRF simulation output from 165 developing and non-developing tropical disturbances. The dataset consisted of small regions of 50×50 gridpoints, extracted from the simulation output according to the disturbance location provided by the IMD best track data. Each datapoint of the dataset had multiple features, and spread across multiple timesteps. Datapoints were labelled to belong to the positive class if they were extracted from developing disturbances, while others were labelled with the negative class. A total of 5560 datapoints were obtained with almost equal class ratio. The model was designed using both the Convolutional Long Short-term Memory (ConvLSTM) [33] layer and the Convolutional Neural Network (CNN) [15] layer, and was implemented using Keras [34]. The first layer was a ConvLSTM layer, followed by a pooling layer, then a dropout layer for regularization, feeding into a convolutional layer and subsequently into a couple of fully-connected dense layers.

A 80 : 20 train-test ratio was used. The obtained probability of detection (POD) was $\sim 99\%$ over the test data. However, this model had low probability of detection (less than 30%) when applied to the entire simulation output (the whole application domain) for multiple disturbances, with a high false alarm ratio. This indicated that the underlying nature of the data across the entire grid was very different from the one that the model was trained on (extracted regions). Therefore we attempted to design a framework that can handle large simulation output (entire domain). Note that simulation output may be in the range of gigabytes depending on the resolution. Thus we designed a two-tier architecture for our framework – a pre-filtering mechanism was used to reduce the candidate regions, and a model was trained on the filtered regions for accurate predictions of cyclogenesis. We explain this next.

B. Prediction Framework

Figure 3 depicts the overall structure of our prediction framework. It consists of three major components – the trigger mechanism, the deep predictive model and an user interface for monitoring cyclogenesis events predicted by the framework. WRF simulation output (history output) forms the input data for our framework which executes simultaneously with an

ongoing WRF simulation. The framework analyzes the output generated at every history interval. The framework processes

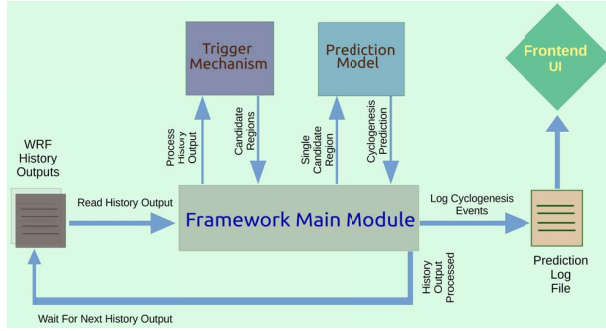


Fig. 3: Tropical Cyclogenesis Prediction Framework

every output and determines whether there is any possibility of a cyclone (via triggers, explained next). In case of positive triggers/alerts, we employ deep predictive model to detect cyclogenesis. The framework event monitoring UI displays the list of cyclogenesis events detected and is updated regularly as the simulation progresses.

1) *Trigger Mechanism*: The trigger mechanism determines whether there are regions of disturbance (i.e. regions of interest) that may mature to cyclones and selects these potential regions. Deep predictive model is then applied to these filtered regions. Applying the deep predictive model over the entire grid of the simulation output is computationally expensive and may lead to a higher false alarm rate. The trigger function lowers the number of potential cyclonic regions to be fed to the deep learning model for accurate predictions.

Algorithm 1: Average Pooling (used in trigger function)

Input : InputGrid

Data: KSize

Result: Average Pooled Data Grid

```

1 PadGrid ← Pad the InputGrid with KSize-1 copies of
  the edge values
2 nrows ← Number of rows in InputGrid
3 ncols ← Number of columns in InputGrid
4 PooledGrid ← Empty grid having dimensions of
  InputGrid
5 for r ← [1...nrows] do
6   for c ← [1...ncols] do
7     PooledGrid[r][c] ←
       
$$\frac{\sum_{i=0}^{KSize-1} \sum_{j=0}^{KSize-1} PadGrid[r+i][c+j]}{KSize^2}$$

8     PooledGrid[r][c] ←  $\frac{PooledGrid[r][c]}{KSize^2}$ 
9   end
10 end
11 return PooledGrid

```

Algorithm 1 is a helper function used by the trigger mechanism. It takes a grid of values as input, and a pre-determined averaging square kernel of size KSize as a parameter. The value of KSize, based on the achieved probability of detection, was empirically chosen to be 4 gridpoints. The input grid

is padded with $KSize - 1$ copies of the values at the edges of the grid. At each grid point coordinate (r, c) of the input grid, a $KSize \times KSize$ region is extracted from the padded grid at that coordinate. The average of the values in the region is calculated, and the value is inserted into the output pooled grid (lines 7–8). After the entire grid is computed, an average pooled output grid with the same dimensions as the input grid, is obtained. This average pooled grid is the output grid data.

Algorithm 2 describes thresholding-based trigger function. It considers U10, V10, AVO and SLP (see Table I) from the simulation output file. Wind speed is calculated from U10 and V10. Then it calculates the average pooled grids of the wind speed, absolute vorticity and sea-level pressure using Algorithm 1 (lines 2–4). Next, for each grid point of the pooled grid, it checks whether the value satisfies the given thresholds (lines 12 – 15). If the value satisfies the thresholds, then it appends the index of the grid point to the output array (line 16). Once the entire output corresponding to a time step is processed, it returns the array to the framework’s main module for more analysis. Note that we consider only the grid points that lie between latitude $5^\circ N$ and $25^\circ N$ (line 11) because tropical cyclones are formed between these latitudes [4]. The variables considered here are the important variables to determine cyclogenesis as mentioned in prior works (see Section III). Thus they form an effective filter to select potential regions.

The thresholds $SlpThresh$, $WindThresh$, $AvoThresh$ and $NThresh$ correspond to the values of the maximum sea-level pressure threshold, minimum wind speed, minimum low-level absolute vorticity and the minimum number of neighbours of a gridpoint with higher sea-level pressure than the grid point respectively. $NThresh$ has been defined in this way to locate grid points that have lower SLP than the neighbours, thereby indicating some possibility of cyclone formation. However, $NThresh$ alone did not suffice as an effective filter. Therefore we considered the other important variables as well. $SlpThresh$, $WindThresh$, $AvoThresh$ and $NThresh$ were set to 1050 HPa, 5 ms^{-1} , 4 s^{-1} and 4 neighbours respectively. These values were empirically determined to produce the best probability of detection (POD) with the lowest possible false alarm ratio (FAR). §V describes the various experiments that were used to determine these thresholds. Probability of detection (POD) value of 96.76% was obtained using these threshold values, with a false alarm ratio (FAR) value of 68.52%. This indicates that the trigger function was able to detect 96.76% (POD) of the tropical disturbances while 68.52% (FAR) of the total detected regions (alerts) by the trigger were false alarms. The deep predictive model performs further analysis of these detected regions to discriminate tropical cyclones from the non-developing tropical disturbances, and to lower the FAR. We applied our initial model [31] to the dataset produced by Algorithm 2 but it did not produce good results. Thus we built a DL model for the regions of interest generated by Algorithm 2.

2) *Deep Learning Model*: The proposed DL model is based on the convolutional neural networks. Figure 4 depicts the

Algorithm 2: Trigger Mechanism

Input: U10, V10, AVO, SLP from simulation output

Data: RegionSize, SlpThresh, WindThresh, AvoThresh, NThresh

Result: Array of coordinates of potential cyclogenesis centers

```
1 WindSpeed  $\leftarrow \sqrt{U10^2 + V10^2}$ 
2 PooledWind  $\leftarrow$  Average Pooling of WindSpeed
3 PooledAVO  $\leftarrow$  Average Pooling of AVO
4 PooledSLP  $\leftarrow$  Average Pooling of SLP
5 nrows  $\leftarrow$  Number of rows in U10/V10/AVO/SLP
6 ncols  $\leftarrow$  Number of columns in U10/V10/AVO/SLP
7 OutArr  $\leftarrow$  Output array of potential cyclogenesis centers,
  initially empty
8 for row  $\leftarrow [1 \dots nrows]$  do
9   for col  $\leftarrow [1 \dots ncols]$  do
10    (Lat, Lon)  $\leftarrow$  Latitude and longitude of
      (row, col)
11    if Lat  $\geq 5^\circ N$  and Lat  $\leq 25^\circ N$  then
12      if PooledSLP[row][col] < SlpThresh then
13        if PooledWind[row][col] >
          WindThresh then
14          if PooledAVO[row][col] >
            AvoThresh then
15            if PooledSLP[row][col] lower
              than at least NThresh neighbours
              then
16              OutArr.append(row, col)
17            end
18          end
19        end
20      end
21    end
22  end
23 end
24 return OutArr
```

structure of the proposed deep predictive model. We describe the layers next. The first layer is the CNN layer, into which the input features of the output grid is fed. Layer 1 contains 128 filters with a kernel size of 3×3 , generating 128 feature maps. These feature maps are then fed into a max pooling layer. Dropout layer is the third layer, responsible for providing regularization, to prevent over-fitting. Then the output is passed into a convolutional layer having 256 filters with same kernel size as layer 1. Layer 5 and layer 6 have 256 filters, and the same kernel size as previous convolutional layers. Layer 7 has 512 filters, thus doubling the number of feature maps. Layer 8 flattens the feature maps into a 1D array of nodes. The flattened layer is connected to a fully-connected dense layer (layer 9) with 1024 nodes. Again dropout is performed for regularization in layer 10. Layers 11, 12, 13, 14, 15 and 16 are all dense layers, with number of nodes equal to 512, 256, 64, 32, 16 and 4 respectively. All the convolutional and dense layers use the Rectified Linear Unit (ReLU) activation function. The final output layer consists of one node, predicting the probability of cyclogenesis, with sigmoidal activation function. Binary cross-entropy is used as the loss function, with stochastic gradient descent (with

learning rate = 0.1) used as the optimizer for training. While testing, the probability generated by the output layer is rounded to the closest integer to obtain the class prediction. The predictive model accurately detects approximately 95% of the tropical cyclones with a FAR of 21.69% over the dataset used in the experiments. Section V describes the experiments performed to arrive at this model architecture, the dataset and selected features used for training the model.

3) *Event Monitoring User Interface (UI)*: Every cyclogenesis event predicted by the model is displayed on an user interface (Figure 5). The UI is continuously updated during the simulation. As the framework generates predictions about probable regions of cyclogenesis, the predictions are dumped to a backend log file. The frontend continuously monitors the log file, and updates itself as the log file is populated by the framework. This enables *on-the-fly* prediction of cyclones (real-time monitoring).

C. Framework Implementation

The framework was mainly implemented in C and we used the NetCDF library [35] for I/O. The functions to calculate derived variables such as absolute vorticity, potential vorticity and relative humidity (derived from QVAPOR) was inspired from the Fortran/Python implementation in the wrf-python [36] library. These derived variables and the other required meteorological variables (shown in Table I) present in the WRF output file are used in our trigger function and deep learning model (details in §V). WRF home directory is input to the framework. It extracts information regarding the current simulation from WRF configuration file (`namelist.input`). After reading the simulation configurations, it starts a main loop that continuously monitors the WRF history output. As soon as a history output is generated, it is sent to the trigger module. The trigger module processes the output data according to Algorithm 2 and generates an array of coordinates. These coordinates are probable candidates for tropical cyclogenesis events. For each coordinate, a 50×50 grid point region is extracted around it, and passed to the DL model.

The information (consisting of multiple features over the region) is passed to a Python script using the Python API from the C code. This script applies the prediction model over it, and returns the prediction result to the C program. The prediction model was implemented in Python using the Keras library with Tensorflow backend. The wrf-python [36] library was used to extract regions of gridpoints (termed as datapoints) from the WRF output data. These datapoints were adapted into multidimensional Python arrays for training and testing the deep learning models using the wrf-python library. After training and testing various models with different parameter values, the best performing model was stored for usage in the framework. The framework interfaces with the chosen model using the Python API for C for cyclogenesis predictions.

When the deep predictive model predicts an event of cyclogenesis from the data (that was passed by the trigger function), this information (i.e. prediction about cyclogenesis) is recorded in a log file. Information such as the location

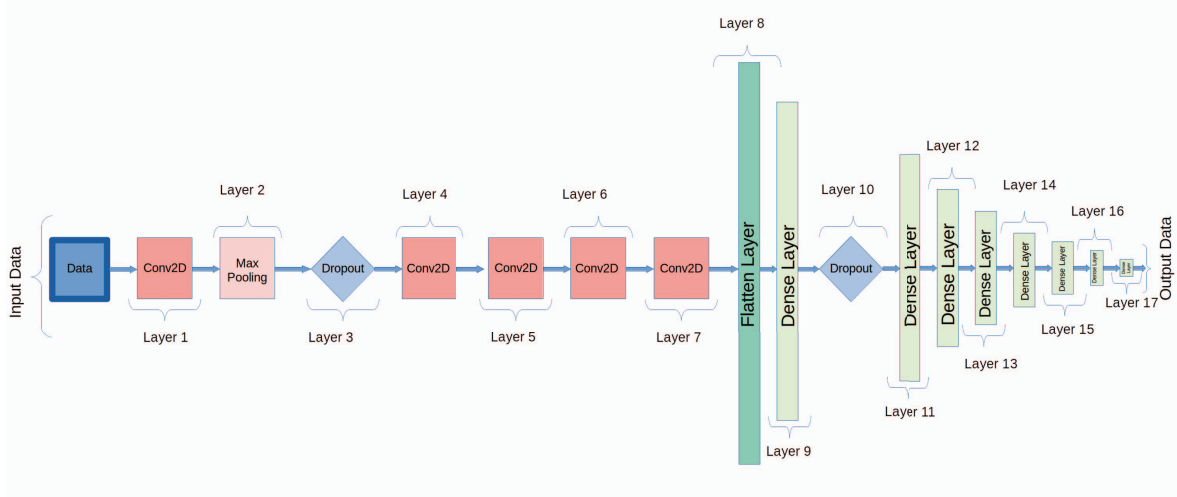


Fig. 4: Deep Learning Model for Tropical Cyclogenesis Prediction

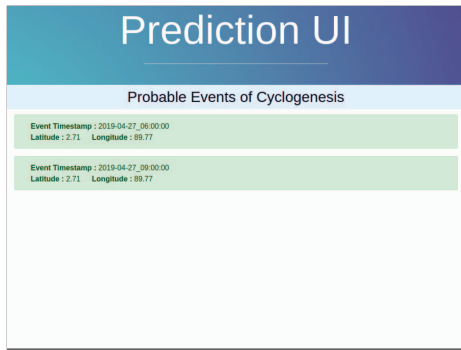


Fig. 5: Event monitoring user interface displays the probable cyclogenesis events to the user.

(latitude, longitude) and the time of event are logged. This log is monitored by the event monitoring user interface (UI). The UI displays the cyclogenesis events for the end user (see Figure 5). The event monitoring UI has two components – a backend server and a frontend web page. The backend server contains routines to dynamically serve the frontend web page with updated content. We implemented this in Python using the Flask library. The frontend webpage was implemented in HTML5 with Bootstrap library. The webpage continuously queries the server every 2 seconds, using jQuery and Ajax, to obtain updated information from the backend prediction log. The frontend data is within 1 MB for 1 simulation day.

V. EXPERIMENTS AND RESULTS

We now describe the configurations, the experiments and the results. We also describe the various parameter values tested in the process of designing the proposed framework.

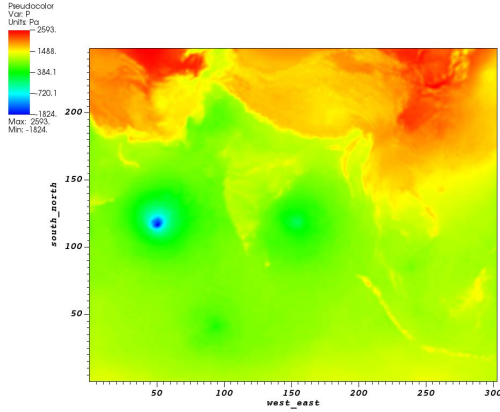
A. Dataset

Weather Research and Forecasting (WRF) numerical weather prediction model simulation output is used as the

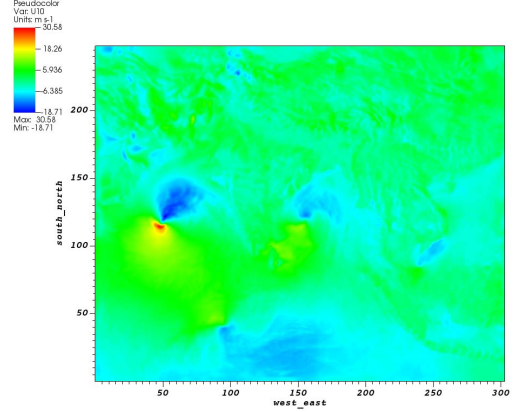
dataset. WRF simulates various atmospheric variables using equations governing the physical interactions in the atmosphere. We chose to simulate 165 tropical disturbances using WRF to create the training and test data. These disturbances were chosen over a period of two decades starting from the year 2000. We obtained the observation data for the WRF simulations from the NCEP FNL Operational Model Global Tropospheric Analyses [37] dataset. We performed the WRF simulations (24 km resolution) using 24 – 32 OpenMP threads (1 MPI process) around the dates of these disturbances (details below) [10] on two Intel-based systems (an Intel Core i9-9900X node (Skylake) and an Intel Xeon Gold 5215 node (Cascade Lake)). We noticed poor scalability of WRF beyond this process/thread configuration on these systems. These simulations resulted in an approximately 500 GB sized dataset.

We define a tropical disturbance to undergo cyclogenesis if it reaches the minimum intensity for being classified as a cyclonic storm by the IMD best track data. The best track data of tropical cyclones in the North Indian Ocean region was obtained from the Indian Meteorological Department [10]. We define change date as the date on which IMD classifies the disturbance as a cyclonic storm. The date from which the IMD track starts for a tropical disturbance is termed as the track start date. The simulations were performed for the selected tropical disturbances from 4 days prior to the track start date to 2 days after the change date. For disturbances that do not have change dates, the simulation was performed for 4 days from the track start date. The length of simulations ranged from 6 – 10 days.

Figure 1 depicts the geographic region that we used as the domain of all the simulations. The domain is centered on longitude 85°E and on latitude 15°N. The grid size of the domain is 305 × 250 (West-East × South-North extent), where each grid point is spaced 24 km apart. The Mercator projection is used as the map projection. Figure 6 shows the visualizations (pseudocolor plot) of two atmospheric variables (perturbation pressure and x-axis wind speed) of cyclone Titli



(a) Low-level Perturbation Pressure (P)



(b) Wind speed along x-axis (U10) at 10m height

Fig. 6: Visualizations of perturbation pressure and x-direction wind speed of cyclone Titli on 9th October, 2018 at 00 hours.

using VisIt [14]. The history interval (output frequency) for the simulations was set to 3 hours (this is the default). Each output frame required approximately 75 MB of storage. The total simulation output was ~ 5 GB per simulation run per cyclone, and 500 GB for all cyclones that were simulated in this work. We have used a coarser resolution of 24 km for simulation for lower execution times. However, when lower (finer) resolutions are used for more accurate simulation output, terabytes of data are generated. We also empirically noted that the prediction accuracy was not affected much by increasing resolution. Our framework will seamlessly work for such cases as well due to our novel trigger mechanism that acts as a filter to the deep learning model and helps in pruning the data that is fed to the model.

B. Trigger Mechanism and Thresholds

The trigger mechanism (Section IV-B1) filters out probable cyclonic regions (positive regions) when the simulations execute. We denote this as online detection of cyclones, also referenced as trigger in the literature [32]. The accuracy/performance of the trigger is determined based on the IMD best track data. The trigger filter was developed using multi-day WRF simulation data of 165 tropical disturbances. This mechanism tested regions centered on each gridpoint for being probable candidates for cyclogenesis. The regions were labelled according to the best track data of tropical cyclones obtained from IMD [10]. The regions were set to 50×50 grid points, corresponding to an area of 1200×1200 sq. km. A region of this size ensured that in case of discrepancies between WRF simulated location and the best track location, the centre of the cyclone will not fall outside of the region boundaries. If a region of this dimension consists of the eye of the tropical disturbance, as provided in the IMD best track data, then that region was labelled as a positive region. If a region did not overlap with the tropical disturbance center, then it was treated as a negative region. In Figure 7, the red boxes denote positive regions while the white boxes demonstrate examples of negative regions. The primary objective of the

trigger mechanism was to detect the positive regions. The positive region may be a developing or a non-developing disturbance, and this analysis is done by the deep predictive model. The trigger may produce multiple overlapping candidate regions, as in Figure 7, because the regions centered on neighbouring gridpoints may have similar properties. By definition, if they contain the eye of the disturbance, they are treated as accurate positive predictions.

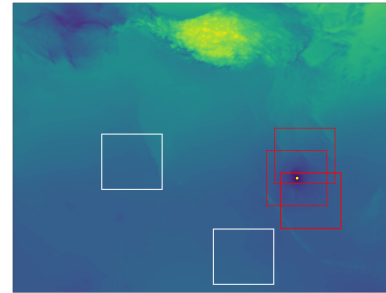


Fig. 7: Positive (red boxes) and negative (white boxes) regions in the WRF output based on the IMD best track data.

The trigger mechanism was designed using thresholding functions as they are both computationally inexpensive and can act as a good pre-filter for the deep learning model. These functions were based on the conditions defined by Gray et al. [7]. Other works [27] (see Section III) provided insights into the significant variables affecting cyclogenesis. Thus, we arrived at three potential features as thresholding parameters – wind, sea-level pressure (SLP) and absolute vorticity as described in Section IV-B1. These were used to identify the tropical disturbances. Additionally, an average pooling was performed on all the chosen features to smooth out any isolated outlier gridpoints. The threshold for SLP was determined by calculating a cumulative histogram of the minimum SLP in a region across all the regions that are centered on the cyclone eye location in the simulation dataset. Figure 8 demonstrates that the value of 1050 HPa has a cumulative frequency

covering almost the entire dataset of 5560 datapoints. Hence, it was selected as the value of $SlpThresh$ in Algorithm 2.

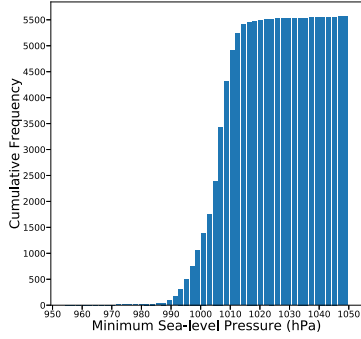


Fig. 8: Cumulative histogram of the minimum SLP of tropical disturbances over the entire dataset of 5560 cyclonic regions (extracted from simulations of 165 disturbances).

The parameter $NThresh$ of Algorithm 2 is the threshold on the number of neighbours having sea-level pressure greater than that of the grid point under consideration. Cyclones have low pressure centres surrounded by high pressure areas. Thus if a grid point is surrounded by higher pressure regions, then that grid point has a higher chance of being the centre of a tropical disturbance. The threshold parameter $WindThresh$, denoting the minimum value of wind speed that a grid point must have, was chosen because tropical disturbances have high speed winds. Absolute vorticity threshold parameter, $AvoThresh$, was chosen on the basis of the Gray conditions. The values of these three parameters were determined experimentally, as shown in Figure 9, to obtain the best performing set of threshold values. This is performed by testing different values of the threshold parameters in the algorithm, and applying it over the dataset of 165 multi-day tropical disturbance simulations. The candidate date for thresholding is obtained by analyzing the values around cyclonic and non-cyclonic regions. For each set of parameter values, a probability of detection (POD) and false alarm ratio (FAR) value is obtained, and the set of values producing the highest POD with the lowest FAR are chosen as the threshold values. $WindThresh$ value of $5m.s^{-1}$, $AvoThresh$ value of $4s^{-1}$ and $NThresh$ value of 4 neighbours, produced the highest average POD of 96.76% and the lowest FAR of 68.52% on an average. The metrics were calculated using IMD best track data.

C. Deep Learning Model Training and Results

The primary objective of the deep learning model is to accurately predict the regions of potential cyclogenesis events from the pre-filtered regions generated by the trigger mechanism. We generated a dataset of all the regions predicted by the trigger mechanism over 30 randomly chosen tropical disturbance simulation output, including the true regions. This resulted in a dataset of total size of around 3 GB containing approximately 60,000 datapoints for training and testing our proposed models. Each datapoint is a region of 50×50 grid points, centered around the corresponding grid point predicted

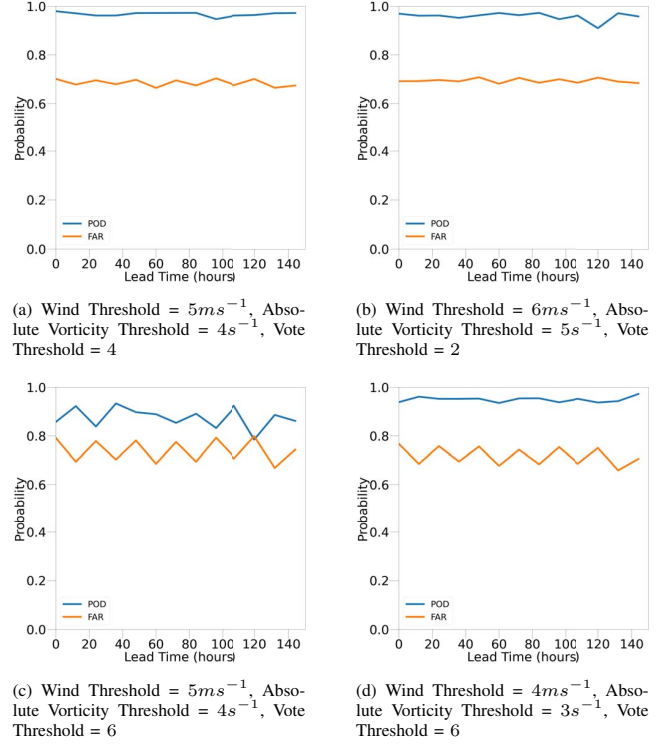


Fig. 9: Performance of trigger mechanism (Algorithm 2) for different threshold values.

by the trigger mechanism. The datapoints were assigned class labels using the best track data. The positive regions consisting of disturbances that develop into cyclonic storms, are labelled with the positive class. Rest are labelled with the negative class. A 80 : 20 split of the dataset was used for training and testing the models respectively.

We designed the cyclogenesis prediction model using convolutional networks. Figure 4 shows the architecture of the proposed deep predictive model. We selected five features $V10$ (wind along y-axis at 10 m height), potential vorticity, absolute vorticity, relative humidity and sea-level pressure. This choice of variables was based on the conditions described in [7]. The model was trained on the dataset for 25 epochs with 10% of the training data used for validation. The validation was done using Keras [34] “validation_split” parameter, while fitting the model over the dataset. This ensured that the validation data is extracted before training, and then validation accuracy is tested at each epoch. Figure 10 shows the training accuracy and the validation accuracy over the training epochs. Training for higher epochs led to over-fitting, and thus gave a reduced test accuracy. We increased both the number of layers and the number of filters per layer as compared to our initial approach (Section IV-A). Deep model architecture was used to capture more abstract features, whereas large number of filters were used to capture more latent features from the input data.

POD is the ratio of the cyclonic regions correctly detected to the total number of cyclonic regions. FAR(false alarm ratio) is the ratio of the falsely predicted cyclonic regions to the

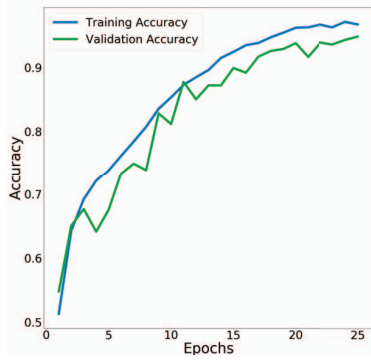


Fig. 10: Analysis of training vs. validation accuracy over number of epochs for 9 : 1 training-validation split.

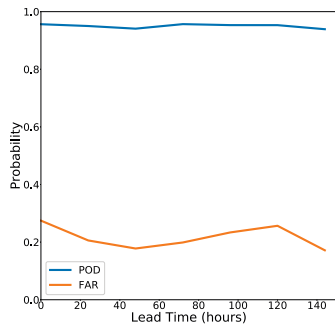


Fig. 11: Prediction performance of the framework over 30 tropical disturbances.

total number of predictions. The proposed model achieves a good average probability of detection (POD) of approximately 95% and an average false alarm ratio (FAR) value of 21.69% over the test dataset. This is achieved with a lead time of up to 150 hours from the change date, as defined previously. Figure 11 shows the POD and FAR values with increasing lead times (x-axis). Matsuoka et al. [29] obtains a POD of approximately 80% with FAR higher than 32%, with similar lead times using deep learning based ensemble model. Zhang et al. [30] trained a deep learning model on tropical cloud cluster data, this led to 97% POD with 20.3% FAR value, but with lead times of 24 – 120 hours. Thus, we observe that our proposed model gives comparable results, and is able to predict tropical cyclogenesis events with high accuracy and high lead times.

D. End-to-end Setup and Timing Profile

We analyzed the timing characteristics of different important components of the framework on two different computing systems – cseproj1 and cseproj2. cseproj1 has Intel(R) Core(TM) i9-9900X CPU @ 3.50GHz processor with 10 hyper-threaded cores and 96 GB memory. cseproj2 has two Intel(R) Xeon(R) Gold 5215 CPU @ 2.50 GHz processors, each with 10 hyper-threaded cores and 125 GB total memory. A RAMFS (in-memory filesystem) was mounted on the physical RAM and the framework was stored in it for faster access. WRF simulations were setup on both these systems for real-

time predictions. We used 32 OpenMP threads on cseproj1 while 24 OpenMP threads were used on cseproj2. We noticed poor scalability of WRF beyond this process/thread configuration on these systems. The simulations for generating the source dataset of 165 cyclones were also performed on cseproj1 and cseproj2 (used simultaneously for quicker generation of data). The simulations were run from the year 2000 – 2019. It took approximately 100 seconds to generate a single history output on cseproj1, while it took almost 210 seconds on cseproj2.

One of the primary requirements and features of the framework is efficiency. The framework needs to be able to process the simulation output quickly. The average time taken to read the entire grid of a 2D variable was approximately 0.2 milliseconds and 2.3 milliseconds on cseproj1 and cseproj2 respectively. The time taken by the trigger mechanism to process a single history output was approximately 1.2 milliseconds on cseproj1 and 2 milliseconds on cseproj2. The time taken for process-to-process transfer of a region of 50×50 gridpoints from the trigger, with five features, to the Python interpreter took around 0.3 milliseconds on cseproj1 and around 0.5 milliseconds on the cseproj2 system. As noted from the above timings, the time taken by the framework to process an entire history output is in the order of milliseconds. This shows that the time to analyze an history output in our framework is insignificant in comparison to the time taken to generate the WRF history output. Thus, this is an efficient framework for real-time cyclogenesis predictions.

It took approximately 220+ combined hours of simulation runtime to generate the entire dataset of 7400 output data frames. We ran the simulations and trained using the simulated output data because we did not find a comprehensive repository of data for training. The same model can be used without further training to predict future cyclones with up to 150 hours of lead times. The deep learning model was trained and tested on a computing system with Intel(R) Core(TM) i7-7700HQ CPU @ 2.80GHz processor, with 4 cores capable of running 8 threads, and a memory of 16 GB. The system also has a CUDA capable NVIDIA 1050Ti GPU, which has a 6.1 NVIDIA compute capability. The training of the model took approximately 51 s per epoch, whereas it took less than a millisecond on average to predict a single data point. Thus our framework is an efficient real-time cyclogenesis predictor.

VI. CONCLUSIONS AND FUTURE WORK

We presented an integrated online framework for predicting tropical cyclogenesis using deep learning. The framework consisted of two major modules – a trigger mechanism and a deep predictive model. The trigger mechanism was designed using thresholds deduced from the properties of tropical cyclones. It was tested over 165 developing and non-developing tropical cyclones, across multiple days of simulation per cyclone. The best track data from IMD was used for labelling the regions. Trigger mechanism was able to detect the tropical disturbances, which may be candidates for cyclogenesis, with a probability of detection (POD) value of 96.76%. The false

alarm ratio had a value of 68.52%, of all the regions predicted by the trigger. We designed a convolutional neural network based model to further identify the true regions of tropical cyclogenesis, from the regions predicted by the trigger function. We obtained average testing POD of 95% and the average FAR was 21.69% using the DL model. The lead time of these predictions were up to 150 hours. The average time taken by the trigger mechanism to process a history output from two different storage systems (RAMFS and hard disk drive) is in the range of 1 – 2 milliseconds. The prediction of each datapoint by the DL model took less than 1 ms on average.

In future, we would like to explore more meteorological insight to achieve a higher accuracy in the triggering mechanism. A more robust implementation may help eliminate the redundant predictions of overlapping regions. We plan to improve the event monitoring user interface to make it more informative for the end user. We also plan to add online visualization of the simulation with integrated prediction information to the user interface. We also plan to integrate other adverse weather events such as flash floods into the framework.

VII. ACKNOWLEDGEMENTS

This work is supported by the Science and Engineering research board (SERB), Department of Science and Technology (DST), Government of India under the SERB Start-up Research Grant (SRG/2020/002503) to Preeti Malakar. We are thankful to Prasanna Balaprakash for initial discussions. We thank IIT Kanpur for reliable systems and networks.

REFERENCES

- [1] N. Anbarci, M. Escaleras, and C. A. Register, "Earthquake fatalities: the interaction of nature and political economy," *Journal of Public Economics*, vol. 89, no. 9, pp. 1907–1933, 2005.
- [2] B. Mallick and et al., "Coastal livelihood and physical infrastructure in Bangladesh after cyclone Aila," *Mitigation and Adaptation Strategies for Global Change*, vol. 16, no. 6, pp. 629–648, 2011.
- [3] S. Goswami and et al., "A review on application of data mining techniques to combat natural disasters," *Ain Shams Engineering Journal*, vol. 9, no. 3, pp. 365–378, 2018.
- [4] B. H. Tang and et al., "Recent advances in research on tropical cyclogenesis," *Tropical Cyclone Research and Review*, vol. 9, no. 2, pp. 87–105, 2020.
- [5] M. Kim, M.-S. Park, J. Im, S. Park, and M.-I. Lee, "Machine learning approaches for detecting tropical cyclone formation using satellite data," *Remote Sensing*, vol. 11, p. 1195, 2019.
- [6] M. S. Peng, B. Fu, T. Li, and D. E. Stevens, "Developing versus Nondeveloping Disturbances for Tropical Cyclone Formation. Part I: North Atlantic," *Monthly Weather Review*, vol. 140, no. 4, 2012.
- [7] W. M. Gray, "Global view of the origin of tropical disturbances and storms," *Monthly Weather Review*, vol. 96, no. 10, pp. 669–700, 1968.
- [8] C. Skamarock and et al., "A Description of the Advanced Research WRF Model Version 4 (No. NCAR/TN-556+STR)," NCAR, Tech. Rep., 2019.
- [9] W. M. Gray, "Tropical cyclone genesis," Ph.D. dissertation, Colorado State University, Fort Collins, CO 80523, USA, 1975.
- [10] IMD, "Indian Meteorological Department - Best Track Data," 2021. [Online]. Available: http://www.rsmcnwdelhi.imd.gov.in/report.php?internal_menu=MzM=
- [11] UCAR, "Research Data Archive," <https://rda.ucar.edu/>, 2022.
- [12] W. M. Gray, "Hurricanes: Their formation, structure and likely role in the tropical circulation," *Meteorology over the tropical oceans*, vol. 155, p. 218, 1979.
- [13] S. Li, S. Jaroszynski, S. Pearse, L. Orf, and J. Clyne, "VAPOR: A Visualization Package Tailored to Analyze Simulation Data in Earth System Science," *Atmosphere*, vol. 10, no. 9, 2019.
- [14] H. Childs and et al., "VisIt: An End-User Tool For Visualizing and Analyzing Very Large Data," in *High Performance Visualization*. Chapman & Hall/CRC, 2012, pp. 357–372.
- [15] Y. LeCun and et al., "Backpropagation applied to handwritten zip code recognition," *Neural computation*, vol. 1, no. 4, pp. 541–551, 1989.
- [16] A. Tsuboi and T. Takemi, "The interannual relationship between MJO activity and tropical cyclone genesis in the Indian Ocean," *Geoscience Letters*, vol. 1, no. 1, pp. 1–6, 2014.
- [17] S. Yokota, H. Niino, and W. Yanase, "Tropical cyclogenesis due to ITCZ breakdown: Idealized numerical experiments and a case study of the event in July 1988," *J. Atmos. Sci.*, vol. 72, no. 9, pp. 3663–3684, 2015.
- [18] Z. Wang, "What is the key feature of convection leading up to tropical cyclone formation?" *Journal of the Atmospheric Sciences*, vol. 75, no. 5, pp. 1609–1629, 2018.
- [19] D. J. Halperin and et al., "An evaluation of tropical cyclone genesis forecasts from global numerical models," *Weather and Forecasting*, vol. 28, no. 6, pp. 1423–1445, 2013.
- [20] K. A. McDonnell and N. J. Holbrook, "A Poisson regression model of tropical cyclogenesis for the Australian–southwest Pacific Ocean region," *Weather and Forecasting*, vol. 19, no. 2, pp. 440–455, 2004.
- [21] C. C. Hennon and J. S. Hobgood, "Forecasting tropical cyclogenesis over the Atlantic basin using large-scale data," *Monthly Weather Review*, vol. 131, no. 12, pp. 2927–2940, 2003.
- [22] N. Jaiswal and C. M. Kishtawal, "Prediction of tropical cyclogenesis using scatterometer data," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 49, no. 12, pp. 4904–4909, 2011.
- [23] J. Albert, B. Sahoo, and P. K. Bhaskaran, "Tropical cyclogenesis identification using eddy detection technique for the Bay of Bengal Basin," *Atmospheric Research*, p. 105670, 2021.
- [24] A. Okubo, "Horizontal dispersion of floatable particles in the vicinity of velocity singularities such as convergences," in *Deep Sea Research and Oceanographic Abstracts*, vol. 17. Elsevier, 1970, pp. 445–454.
- [25] J. Weiss, "The dynamics of enstrophy transfer in two-dimensional hydrodynamics," *Physica D: Nonlinear Phenomena*, vol. 48, no. 2–3, pp. 273–294, 1991.
- [26] W. Zhang and et al., "Discriminating developing versus nondeveloping tropical disturbances in the western North Pacific through decision tree analysis," *Weather and Forecasting*, vol. 30, no. 2, pp. 446–454, 2015.
- [27] M.-S. Park and et al., "Detection of tropical cyclone genesis via quantitative satellite ocean surface wind pattern and intensity analyses using decision trees," *Remote sensing of environment*, vol. 183, pp. 205–214, 2016.
- [28] T. Zhang and et al., "Prediction of tropical cyclone genesis from mesoscale convective systems using machine learning," *Weather and Forecasting*, vol. 34, pp. 1035–1049, 2019.
- [29] D. Matsuoka, M. Nakano, D. Sugiyama, and S. Uchida, "Deep learning approach for detecting tropical cyclones and their precursors in the simulation by a cloud-resolving global nonhydrostatic atmospheric model," *Progress in Earth and Planetary Science*, vol. 5, no. 1, pp. 1–16, 2018.
- [30] R. Zhang and et al., "Predicting Tropical Cyclogenesis Using a Deep Learning Method From Gridded Satellite and ERA5 Reanalysis Data in the Western North Pacific Basin," *IEEE Transactions on Geoscience and Remote Sensing*, 2021.
- [31] A. Mukherjee and P. Malakar, "A Deep Predictive Model for Tropical Cyclogenesis," in *2020 International Conference on High Performance Computing Simulation (HPCS)*, 2020.
- [32] J. C. Bennett and et al., "Trigger Detection for Adaptive Scientific Workflows Using Percentile Sampling," *SIAM Journal on Scientific Computing*, vol. 38, no. 5, pp. S240–S263, 2016.
- [33] X. Shi and et al., "Convolutional LSTM Network: A Machine Learning Approach for Precipitation Nowcasting," in *Proceedings of International Conference on Neural Information Processing Systems*, 2015.
- [34] F. Chollet et al., "Keras: the Python deep learning API," <https://keras.io/>, 2021, (accessed August 02, 2021). [Online]. Available: <https://keras.io/>
- [35] R. Rew and G. Davis, "NetCDF: an interface for scientific data access," *IEEE Computer Graphics and Applications*, vol. 10, no. 4, 1990.
- [36] B. Ladwig, "WRF-Python," 2017. [Online]. Available: <https://github.com/NCAR/wrf-python>
- [37] N. National Centers for Environmental Prediction, National Weather Service, "NCEP FNL Operational Model Global Tropospheric Analyses, continuing from July 1999," Boulder CO, 2000.